

task-2

May 17, 2025

```
[5]: # Install necessary packages
!pip install -q nltk pandas scikit-learn matplotlib seaborn

# Imports
import pandas as pd
import nltk
import re
import seaborn as sns
import matplotlib.pyplot as plt

from nltk.corpus import stopwords
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

nltk.download('stopwords')
```

[nltk_data] Downloading package stopwords to /root/nltk_data...

[nltk_data] Package stopwords is already up-to-date!

[5]: True

```
[6]: # Load CSV (make sure this path is correct for your file)
csv_path = "/content/training.1600000.processed.noemoticon.csv"
columns = ['target', 'ids', 'date', 'flag', 'user', 'text']
df = pd.read_csv(csv_path, encoding='latin-1', names=columns)

print("Loaded", len(df), "rows")
df.head()
```

Loaded 1607011 rows

```
[6]:   target      ids      date      flag \
0      0  1467810369  Mon Apr 06 22:19:45 PDT 2009  NO_QUERY
1      0  1467810672  Mon Apr 06 22:19:49 PDT 2009  NO_QUERY
2      0  1467810917  Mon Apr 06 22:19:53 PDT 2009  NO_QUERY
3      0  1467811184  Mon Apr 06 22:19:57 PDT 2009  NO_QUERY
```

4 0 1467811193 Mon Apr 06 22:19:57 PDT 2009 NO_QUERY

	user	text
0	_TheSpecialOne_	@switchfoot http://twitpic.com/2y1zl - Awww, t...
1	scotthamilton	is upset that he can't update his Facebook by ...
2	mattycus	@Kenichan I dived many times for the ball. Man...
3	ElleCTF	my whole body feels itchy and like its on fire
4	Karoli	@nationwideclass no, it's not behaving at all...

```
[7]: # Step 2: Preprocess tweets

# Keep only positive (4) and negative (0)
df = df[df['target'] != 2]
df['target'] = df['target'].map({0: 0, 4: 1})

stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|@\w+|#\w+|[\^a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return " ".join(w for w in text.split() if w not in stop_words)

df['clean_text'] = df['text'].apply(clean_text)

# -----
# Step 3: Train/test split & vectorize
X = df['clean_text']
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

vectorizer = TfidfVectorizer(max_features=10000, stop_words='english')
X_train_tf = vectorizer.fit_transform(X_train)
X_test_tf = vectorizer.transform(X_test)

# -----
# Step 4: Train model
model = LogisticRegression(max_iter=1000)
model.fit(X_train_tf, y_train)
y_pred = model.predict(X_test_tf)

# -----
# Step 5: Evaluate
print("Classification Report:")
```

```

print(classification_report(y_test, y_pred, target_names=["Negative",
↪ "Positive"]))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=["Predicted Neg", "Predicted Pos"],
            yticklabels=["Actual Neg", "Actual Pos"])
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# -----
# Step 6: Analyze sentiment by specific topics

topics = {
    "Politics": ["election", "government", "biden", "trump", "democrat",
↪ "republican", "vote", "president"],
    "Climate": ["climate", "global warming", "co2", "greenhouse",
↪ "sustainability", "carbon", "pollution"],
    "Technology": ["ai", "artificial intelligence", "machine learning", "tech",
↪ "robot", "software", "gadget"],
    "Sports": ["football", "soccer", "cricket", "nba", "tennis", "match",
↪ "goal", "score"]
}

results = []

for topic, keywords in topics.items():
    pattern = '|'.join([fr"\b{k}\b" for k in keywords])
    df_topic = df[df['clean_text'].str.contains(pattern, case=False, na=False)]

    if len(df_topic) < 10:
        print(f"Skipping topic '{topic}' due to insufficient tweets_
↪ ({len(df_topic)})")
        continue

    pos = (df_topic['target'] == 1).sum()
    neg = (df_topic['target'] == 0).sum()
    total = len(df_topic)

    results.append({
        'Topic': topic,
        'Positive': pos,
        'Negative': neg,
        'Total Tweets': total,

```

```

        'Positive %': round(100 * pos / total, 1),
        'Negative %': round(100 * neg / total, 1)
    })

topic_df = pd.DataFrame(results)
print("\nSentiment breakdown by topic:")
print(topic_df)

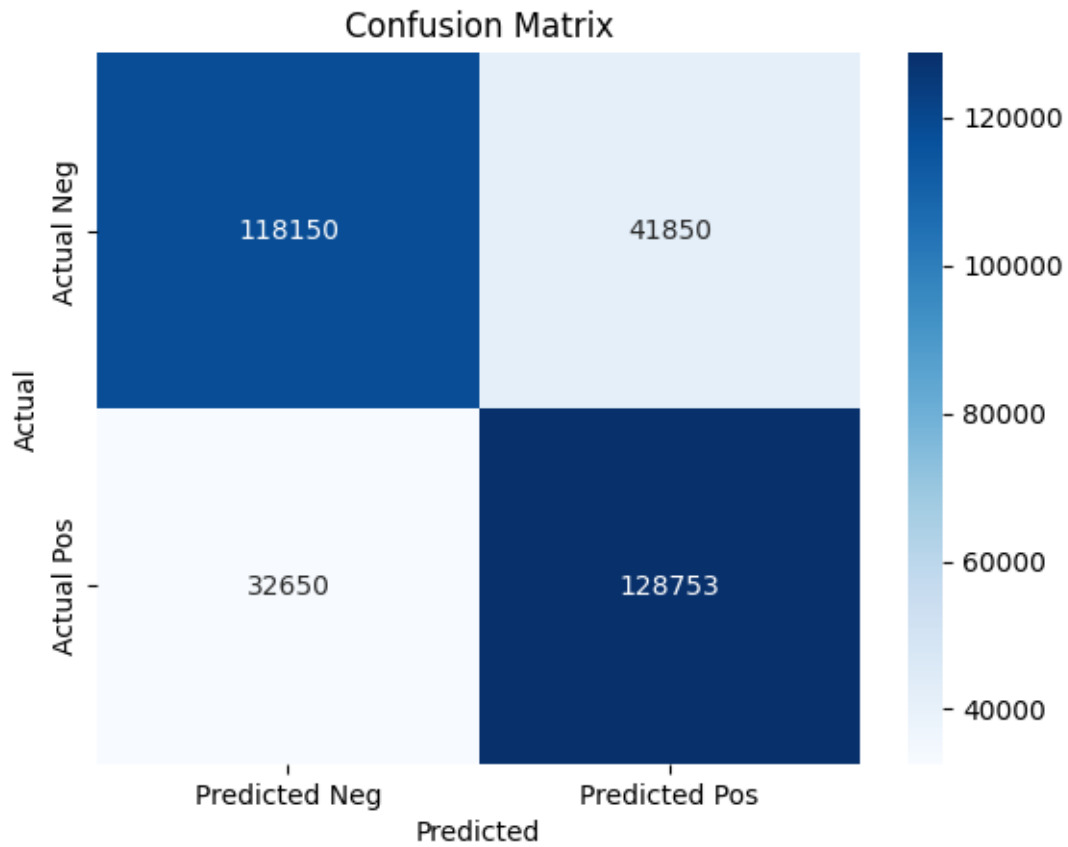
# Plot sentiment counts per topic
plt.figure(figsize=(10, 6))
topic_df_melt = topic_df.melt(id_vars='Topic', value_vars=['Positive', 'Negative'],
                              var_name='Sentiment', value_name='Count')

sns.barplot(data=topic_df_melt, x='Topic', y='Count', hue='Sentiment',
            palette={"Positive": "green", "Negative": "red"})
plt.title("Sentiment by Topic")
plt.ylabel("Tweet Count")
plt.xlabel("Topic")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

Classification Report:

	precision	recall	f1-score	support
Negative	0.78	0.74	0.76	160000
Positive	0.75	0.80	0.78	161403
accuracy			0.77	321403
macro avg	0.77	0.77	0.77	321403
weighted avg	0.77	0.77	0.77	321403



Sentiment breakdown by topic:

	Topic	Positive	Negative	Total Tweets	Positive %	Negative %
0	Politics	1985	1199	3184	62.3	37.7
1	Climate	156	178	334	46.7	53.3
2	Technology	1079	1151	2230	48.4	51.6
3	Sports	2933	2904	5837	50.2	49.8

